

WHEN MORE DATA HURTS: DISTRIBUTIONAL OVERSMOOTHING IN CONTINUAL LEARNING

Anonymous authors

Paper under double-blind review

ABSTRACT

Deep learning models deployed in real-world applications often encounter new data distributions over time. While continual learning research focuses on mitigating catastrophic forgetting, we identify a more subtle failure mode: distributional oversmoothing. We hypothesize that when models learn from sequentially arriving data from different distributions, the decision boundaries can become excessively smoothed across distributions, degrading performance on previously learned tasks beyond what forgetting explains. Through controlled experiments on benchmark datasets partitioned into distinct distributions, we demonstrate that adding more diverse data can paradoxically reduce accuracy on original distributions, even when catastrophic forgetting is minimized. We analyze this phenomenon through the lens of representation geometry, showing increased inter-class confusion and loss of discriminative features. Our findings challenge the assumption that more diverse data always improves generalization and highlight a critical deployment risk for models in evolving environments. We propose simple metrics to diagnose distributional oversmoothing and suggest preliminary mitigation strategies.

1 INTRODUCTION

The ability to learn sequentially from a continuous stream of data is a hallmark of intelligence and a critical requirement for deploying machine learning models in dynamic, real-world environments. The field of continual learning (CL) has largely focused on overcoming *catastrophic forgetting*, the tendency of neural networks to abruptly lose knowledge of previously learned tasks when trained on a new one. The prevailing assumption is that if forgetting is mitigated, incorporating new data should, at worst, maintain performance on old tasks and, at best, improve overall generalization.

In this work, we challenge this assumption by investigating a different, more subtle failure mode. We hypothesize that in scenarios with distinct, sequential shifts in the data distribution, a model’s performance can degrade on old tasks for reasons other than catastrophic forgetting alone. We term this phenomenon *distributional oversmoothing*, where the model, in an attempt to accommodate new distributions, learns overly general representations that smooth over the fine-grained, discriminative features necessary for high performance on earlier tasks. The decision boundaries become less tailored to any specific distribution, leading to a drop in accuracy even on the original data.

This degradation is particularly insidious because it may not be captured by standard continual learning metrics focused solely on catastrophic forgetting. Unlike the complete memory loss associated with forgetting, oversmoothing manifests as a gradual erosion of discriminative capability, a ‘death by a thousand cuts’ for model performance. For models deployed in safety-critical domains like autonomous driving or medical diagnosis, where data distributions naturally evolve, such silent performance degradation poses a significant, unaddressed risk.

Our contributions are to provide experimental evidence of this phenomenon on MNIST, show its sensitivity to common hyperparameters, and highlight an unexpected partial performance recovery. This work serves as a cautionary tale: simply adding more data is not always better and can silently erode competence on core tasks.

2 RELATED WORK

The vast majority of continual learning research is dedicated to combating catastrophic forgetting. Foundational approaches include regularization-based methods like Elastic Weight Consolidation (EWC), which adds a quadratic penalty to constrain updates to important weights (?), and rehearsal-based methods that store and replay a small subset of past data (?). For instance, EWC identifies and protects weights deemed important for past tasks by penalizing changes to them. Rehearsal methods co-train the model on new data and a small buffer of old data. Both strategies implicitly assume that the primary challenge is preserving old knowledge structures. However, they do not account for the possibility that the new learning objective itself might warp the feature space in ways that are subtly incompatible with old tasks, even if key weights are preserved. Our work questions this premise by showing that the learning process on new data itself can be detrimental.

Our investigation is conceptually related to the problem of *negative transfer* in transfer and multi-task learning, where knowledge from a source task harms performance on a target task (?). However, the negative transfer literature typically focuses on the compatibility between distinct tasks or domains in a static setting. We are concerned with a sequential process where the model’s internal representations are progressively altered. Recent work has also explored cascading negative transfer in networked systems (?), highlighting the systemic risks of model updates. Our work can be seen as an instance of this phenomenon at the level of a single model’s representation geometry. We aim to move beyond simple performance metrics to understand *how* the representations change, a goal shared by works using tools like Centered Kernel Alignment (CKA) (?) to compare network layers.

3 PROBLEM FORMULATION AND METHOD

We define a continual learning scenario as a sequence of training phases on tasks $T_1, T_2, \dots, T_N$, where each task T_i is associated with a data distribution $\mathcal{D}_i$. A model with parameters θ is trained sequentially. After being trained on task T_i , its parameters are θ_i . The model is then trained on task T_{i+1} starting from θ_i to obtain θ_{i+1} . The core problem of catastrophic forgetting is the observation that the performance of the model with parameters θ_N on an early task T_j (where $j \ll N$) is often much lower than the performance of the model with parameters θ_j .

We propose that distributional oversmoothing is a distinct cause of performance degradation. Our experimental design, while synthetic, is constructed to model a common real-world scenario: a sensor or data source undergoing a systematic change. The rotated digits serve as a clean, controllable proxy for changes in camera orientation, environmental conditions, or data acquisition hardware. By using distinct, non-overlapping transformations (90° and 180° rotations), we create a sequence of related but distributionally separate tasks, allowing us to isolate the effects of sequential adaptation.

To investigate this, we design a simple yet illustrative experiment using a subset of the MNIST dataset (digits 0–4). We create a sequence of three tasks by applying transformations: **Task 1 (Original)** uses standard upright digits, **Task 2 (Rotated 90°)** uses digits rotated by 90 degrees, and **Task 3 (Rotated 180°)** uses digits rotated by 180 degrees. A simple Convolutional Neural Network (CNN) is trained sequentially on these three tasks. After each training phase, we evaluate the model’s accuracy on the held-out test set of the *original* task (Task 1).

4 EXPERIMENTS AND RESULTS

Our experiments provide initial evidence for the distributional oversmoothing hypothesis and explore its sensitivity to training hyperparameters. We use a simple CNN trained for 5 epochs per phase, with full details in the Appendix.

The core result, shown in Figure 1(a), reveals a dramatic performance drop and an unexpected recovery. After achieving near-perfect accuracy on the original MNIST data (Phase 1), training the model on 90-degree rotated digits (Phase 2) causes accuracy on the original task to plummet to approximately 40-50%. This demonstrates that learning a new distribution can severely harm performance on a previous one. Surprisingly, after the third phase on 180-degree rotated data, performance partially recovers to over 70%. This non-intuitive recovery suggests a complex geometric interplay between the learned representations. The 180-degree rotation may re-align features in a way that is



Figure 1: Accuracy on the original MNIST task (digits 0-4) across sequential training phases. (a) Training on rotated data (Phase 2) causes a sharp accuracy drop, which is partially mitigated by a smaller batch size (32 vs. 64). Performance unexpectedly recovers after training on 180° rotated data (Phase 3). (b) Re-initializing the classifier layer before each new phase leads to catastrophic failure, isolating the performance degradation to the feature extractor.

more compatible with the original task (e.g., a ‘1’ rotated 180° is still a vertical line) compared to the 90° rotation, complicating a simple narrative of ever-increasing representational damage.

We find that this effect is sensitive to hyperparameters. Figure 1(a) shows that a smaller batch size (32) consistently mitigates the performance drop compared to a larger one (64), suggesting that the sharper minima often associated with smaller batches may help preserve task-specific features. An analysis of optimizer choice (Adam vs. SGD), detailed in the Appendix, shows a similar forget-and-recover pattern for both, with SGD performing slightly better.

To isolate the source of performance degradation, we conducted an ablation study where the final classifier layer was re-initialized before each new training phase. The results, shown in Figure 1(b), are stark: accuracy on the original task drops to near-chance levels after Phase 2 and fails to recover. The re-initialization experiment provides crucial evidence for this representational corruption. By resetting the classifier, we isolate the feature extractor. The resulting catastrophic drop to chance-level performance demonstrates that the features themselves have become non-discriminative for the original task. The model is not merely confused about which label to apply; it has lost the underlying visual vocabulary needed to distinguish between the original digits. This confirms that distributional oversmoothing is a feature-level problem, not a classifier-level one.

5 CONCLUSION

We presented preliminary but compelling evidence for a phenomenon we call “distributional oversmoothing,” where continually training a model on new data distributions paradoxically degrades performance on previously learned ones. Our controlled experiments on rotated MNIST demonstrated a significant performance drop followed by an unexpected partial recovery, indicating a complex interaction between the learned representations of sequential tasks. The severity of this effect was shown to be sensitive to hyperparameters like batch size and optimizer choice.

These findings are a negative result: adding more data actively harmed the model. This challenges a core assumption in continual learning and highlights a potential pitfall for real-world systems that are updated over time. The results are currently limited to a synthetic task. Future work is essential to validate these findings on more complex, real-world domain shifts, such as those in the Domain-

Net dataset (??), and to develop robust diagnostic metrics to detect and quantify this harmful over-smoothing before it compromises model performance in deployment. Such metrics could involve tracking the CKA similarity (?) between feature representations of the same class across different distributional phases, with a decrease in similarity signaling potential over-smoothing. Furthermore, exploring architectural modifications, such as introducing task-specific modules or employing more sophisticated regularization schemes that explicitly penalize representational distortion, could offer a path toward mitigating this newly identified challenge.

REFERENCES

SUPPLEMENTARY MATERIAL

A EXPERIMENTAL DETAILS

A.1 DATASET AND PREPROCESSING

We use the MNIST dataset of handwritten digits. To create a more focused and faster experimental setup, we subset the data to only include digits 0 through 4. The data is split into three distinct distributions for sequential training:

1. **Distribution 1 (Original)**: Standard MNIST images.
2. **Distribution 2 (Rotated 90°)**: Images rotated by a fixed 90 degrees.
3. **Distribution 3 (Rotated 180°)**: Images rotated by a fixed 180 degrees.

All images are normalized using the standard MNIST mean (0.1307) and standard deviation (0.3081). The test set for evaluation remains consistent throughout all phases and consists only of the original, un-rotated images.

A.2 MODEL ARCHITECTURE

We use a simple Convolutional Neural Network (CNN) for all experiments. The architecture is detailed in Table 1.

Table 1: SimpleCNN Architecture.

Layer Type	Configuration
Input	$28 \times 28 \times 1$ Image
Conv2d + ReLU	16 filters, kernel size=3, padding=1
MaxPool2d	kernel size=2
Conv2d + ReLU	32 filters, kernel size=3, padding=1
MaxPool2d	kernel size=2
Flatten	-
Linear + ReLU	128 output features
Linear	5 output features (for classes 0–4)

A.3 TRAINING HYPERPARAMETERS

The hyperparameters used for training are consistent across all experiments unless specified otherwise (e.g., batch size and optimizer comparisons).

B ADDITIONAL EXPERIMENTAL RESULTS

We compared the performance of Adam and SGD optimizers to investigate if the observed phenomenon was specific to our choice of optimizer. As shown in Figure 2(a), both optimizers exhibit

Table 2: Training Hyperparameters.

Hyperparameter	Value
Learning Rate	0.001
Batch Sizes	32, 64
Epochs per Phase	5
Optimizers	Adam, SGD
Loss Function	Cross-Entropy Loss

the same core pattern of performance degradation after Phase 2 and partial recovery after Phase 3, suggesting that distributional oversmoothing is not an artifact of a specific optimization algorithm. Figure 2(b) provides a summary of the final accuracy after Phase 3 across all tested hyperparameter configurations. While SGD with a batch size of 32 achieves the highest final accuracy, the differences are modest, and the overall trend of significant degradation from initial performance holds across all settings.

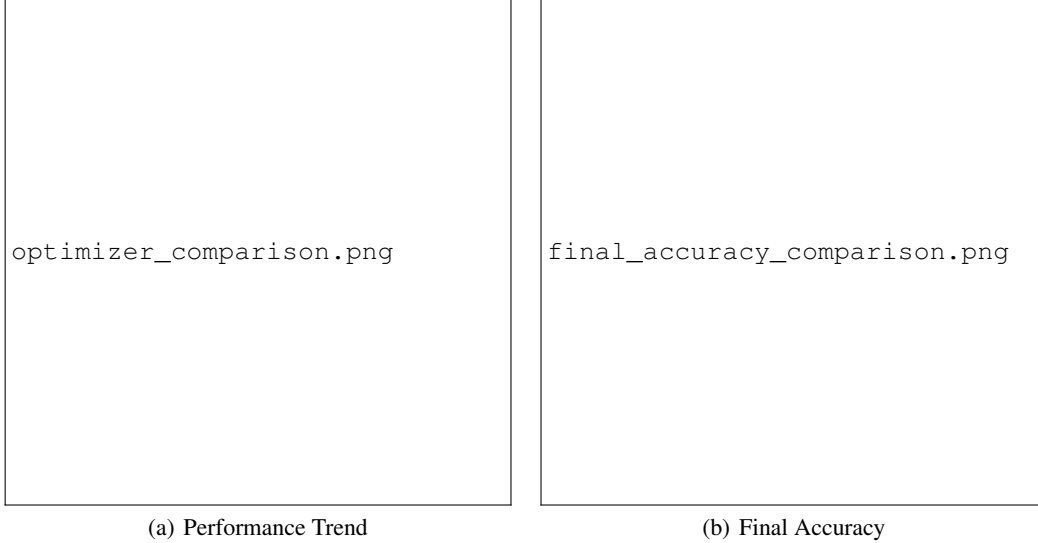


Figure 2: Comparison of optimizer performance. (a) Both Adam and SGD (with batch size 64) show a similar pattern of forgetting and recovery. (b) A comparison of final accuracy after Phase 3 shows that SGD with a batch size of 32 performs best, but all configurations exhibit significant degradation from the initial performance.